Simualtion Inputs

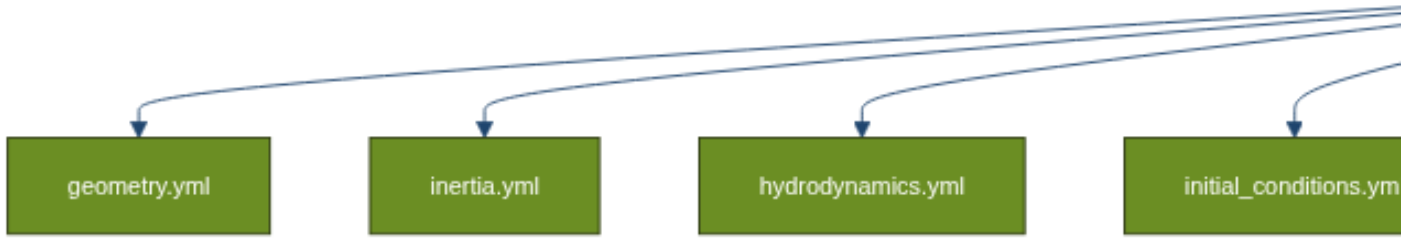
Overview

The Panisim simulator uses a structured set of YAML configuration files to define all aspects of the simulation environment, vessel properties, sensor configurations and guidance. This modular approach allows for easy customization and extension of simulation scenarios without modifying the core simulation code. The input files are auto generated via the interactive Web GUI, but can also be modified manually.

> Tip
>
> All input files follow the YAML format, which is human-readable and easy to edit.

Input File Structure

The input file system follows a hierarchical structure (zoom in to see the chart more clearly).



The top-level `simulation_input.yml` file defines global simulation parameters and references vessel-specific configuration files stored in separate folders (one for each vessel type).

Main Configuration File

The `simulation_input.yml` file is the entry point for the simulator configuration. It specifies global parameters and the list of agents (vessels) to include in the simulation.

Example structure:

```yaml
sim_time: 100.0                # Total simulation time in seconds
time_step: 0.01                # Simulation time step in seconds
density: 1025.0                # Water density (kg/m³)
gravity: 9.80665               # Gravitational acceleration (m/s²)
world_size: [1000, 1000, 100] # Simulation world dimensions [x, y, z] in meters
gps_datum: [13.06, 80.28, 0]  # GPS reference point [latitude, longitude, height]
nagents: 1                     # Number of agents in the simulation
geofence: []                   # Optional geofencing boundaries

agents:
  - name: "sookshma"           # Vessel name
    type: "usv"                # Vessel type (usv, auv, etc.)
    active_dof: [1, 1, 0, 0, 0, 1]  # Active degrees of freedom [u, v, w, p, q, r]
    U: 0.0                     # Initial forward speed
    maintain_speed: false      # Whether to maintain constant forward speed

    # Paths to configuration files (relative or absolute)
    # {name} will be replaced with the vessel name
    geometry: "inputs/{name}/geometry.yml"
    inertia: "inputs/{name}/inertia.yml"
    hydrodynamics: "inputs/{name}/hydrodynamics.yml"
    control_surfaces: "inputs/{name}/control_surfaces.yml"
    thrusters: "inputs/{name}/thrusters.yml"
    initial_conditions: "inputs/{name}/initial_conditions.yml"
    sensors: "inputs/{name}/sensors.yml"
    guidance: "inputs/{name}/guidance.yml"
    control: "inputs/{name}/control.yml"
```

Key Parameters

Parameter	Description	Units
`sim_time`	Total simulation duration	seconds
`time_step`	Simulation time step size	seconds
`density`	Water density	kg/m^3
`gravity`	Gravitational acceleration	m/s^2
`world_size`	Simulation world dimensions [x, y, z]	meters
`gps_datum`	Reference point for GPS coordinates [lat, lon, height]	degrees, degrees, meters
`nagents`	Number of agents (vessels) in the simulation	integer
`geofence`	Optional boundaries for the simulation area	meters

Agent Configuration

Each agent (vessel) in the `agents` list requires:

- `name`: Identifier for the vessel, used for file path resolution
- `type`: Type of vessel ("usv", "auv", etc.)
- `active_dof`: Array of 6 binary values indicating which degrees of freedom are active [surge, sway, heave, roll, pitch, yaw]
- `U`: Initial/desired forward speed
- `maintain_speed`: Boolean flag to maintain constant forward speed

File paths can be absolute or relative (although we would recommend using the same folder structure as the default). The special placeholder `{name}` will be replaced with the vessel name, allowing for a standardized folder structure.

Vessel-Specific Configuration Files

Each vessel has its own set of configuration files that define its physical properties, sensors, and control elements. This is defined in a folder named after the vessel.

Geometry Configuration (`geometry.yml`)

The geometry file defines the vessel's physical dimensions and coordinate frames.

Example from sookshma:

```yaml
# Dimensions of the box enclosing the vessel
length: 1
breadth: 0.24
depth: 0.5
Fn: 0.16
gyration: [0.096, 0.25, 0.25]
CO:
  position: [0.0, 0.0, 0.0] # Relative to the origin of the 3d software
  orientation: [0.0, 0.0, 0.0]
CG:
  position: [2.89415958e-02, 0.0, 1.62558889e-01]
CB:
  position: [2.89415958e-02, 0.0, 1.36000000e-01]

geometry_file: /workspaces/mavlab/inputs/sookshma/HydRA/input/sookshma.gdf
```

Key Parameters

Parameter	Description	Units
`length`	Overall length	meters
`breadth`	Overall width	meters
`depth`	Overall depth	meters
`Fn`	Froude number	dimensionless
`gyration`	Gyration radii [x, y, z]	meters
`CO`	Coordinate origin position and orientation	meters, radians
`CG`	Center of gravity position and orientation	meters, radians
`CB`	Center of buoyancy position and orientation	meters, radians
`geometry_file`	Path to 3D geometry file	string

Inertia Configuration (`inertia.yml`)

The inertia file defines the vessel's mass properties.

Example from sookshma:

```yaml
mass: 7.65 # in kg
buoyancy_mass: 7.65 # in kg
inertia_matrix: None # in kg*m^2
added_mass_matrix: None # in kg*m^2/s
```

Key Parameters

Parameter	Description	Units
`mass`	Vessel mass	kg
`buoyancy_mass`	Mass of displaced fluid (for neutral buoyancy, equal to mass)	kg
`inertia_matrix`	Optional explicit inertia matrix needs to be 6x6 (example: `[[1, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0], [0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 1, 0], [0, 0, 0, 0, 0, 1]]`)	$kg \cdot m^2$
`added_mass_matrix`	Optional explicit added mass matrix needs to be 6x6	kg, $kg \cdot m^2$, $kg \cdot m$

If the inertia matrix is not provided, it is calculated using the gyration radii and the mass via the `_generate_mass_matrix()` function in the `calculate_hydrodynamics.py` module. If the added mass matrix is not provided, it will be calculated either using the hydrodynamic derivatives or using the hydrodynamic coefficients from the HydRA model.

Hydrodynamics Configuration (`hydrodynamics.yml`)

The hydrodynamics file contains coefficients for damping forces and added mass.

Example from sookshma:

```yaml
hydra_file: /workspaces/mavlab/inputs/sookshma/HydRA/matlab/sookshma_hydra.json
dim_flag: false
cross_flow_drag: false
coriolis_flag: false # For linear dynamic model

# Surge hydrodynamic derivatives
X_u: -0.01

# Sway hydrodynamic derivatives
Y_v: -0.010659050686665396
Y_r: 0.0017799664706713543

# Yaw hydrodynamic derivatives
N_v: -0.0006242757399292063
N_r: -0.001889456681929024
```

Key Parameters

Parameter	Description	Units
`hydra_file`	Path to HydRA data file	string
`dim_flag`	Whether coefficients are dimensional (true) or non-dimensional (false)	boolean
`cross_flow_drag`	Whether to use cross-flow drag calculation	boolean
`coriolis_flag`	Whether to include Coriolis forces	boolean

You can enter any combination of the hydrodynamic coefficients, and it'll be handled by the simulator. They should be separated by underscores. For example, `X_u_u` or `M_r_r`.

Control Surfaces Configuration (`control_surfaces.yml`)

This file defines the vessel's control surfaces such as rudders, fins, or foils.

Example from sookshma:

```yaml
naca_number: &naca None
naca_file: &naca_file None
delta_max: &dmax 35.0
deltad_max: &ddmax 1.0
area: &area 0.1

control_surfaces:
  -
    control_surface_type: Rudder
    control_surface_id: 1
    control_surface_location: [0.0, 0.0, 0.0]    # With respect to BODY frame
    control_surface_orientation: [0.0, 0.0, 0.0]  # With respect to BODY frame
    control_surface_area: *area
    control_surface_NACA: *naca
    control_surface_T: 0.1
    control_surface_delta_max: *dmax
    control_surface_deltad_max: *ddmax
    control_surface_hydrodynamics:
      Y_delta: 0.02100751285923063
      N_delta: -0.006813248905845932
      X_delta: 0.0
```

Key Parameters

Parameter	Description	Units
`naca_number`	NACA airfoil designation	string
`naca_file`	Path to NACA airfoil data file	string
`delta_max`	Maximum deflection angle	degrees
`deltad_max`	Maximum deflection rate	degrees/s
`area`	Surface area	m^2
`control_surface_type`	Type of control surface (e.g., Rudder, Fin)	string
`control_surface_id`	Unique identifier	integer
`control_surface_location`	[x, y, z] position	meters
`control_surface_orientation`	[roll, pitch, yaw] orientation	radians
`control_surface_T`	Actuation time constant	seconds
`control_surface_hydrodynamics`	Force coefficients for the control surface	various

If `control_surface_hydrodynamics` is provided, the control surface location, orientation and naca file will be ignored. And the generalized force vector due to control surface will be calculated using the hydrodynamic coefficients.

Thruster Configuration (`thrusters.yml`)

This file defines the vessel's propulsion systems.

Example from sookshma (commented out):

```yaml
thrusters:
  -
    thruster_name: back
    thruster_id: 1
    thruster_location: [0.0, 0.0, 0.0]    # With respect to BODY frame
    thruster_orientation: [0.0, 0.0, 0.0]  # With respect to BODY frame
    T_prop: 1.0
    D_prop: 0.1
    tp: 0.1
    KT_at_J0: 0.0
    n_max: 2668 # in RPM
    J_vs_KT: None # File path to J_vs_KT.csv, Not being used currently
```

Key Parameters

Parameter	Description	Units
`thruster_name`	Descriptive name	string
`thruster_id`	Unique identifier	integer
`thruster_location`	[x, y, z] position	meters
`thruster_orientation`	[roll, pitch, yaw] orientation	radians
`T_prop`	Propeller thrust coefficient	dimensionless
`D_prop`	Propeller diameter	meters
`tp`	Thrust deduction coefficient	dimensionless
`KT_at_J0`	Thrust coefficient at zero advance ratio	dimensionless
`n_max`	Maximum RPM	RPM
`J_vs_KT`	Path to file with advance ratio vs thrust coefficient data	string

Initial Conditions Configuration (`initial_conditions.yml`)

This file defines the vessel's initial state for the simulation.

Example from sookshma:

```yaml
start_location: [0, 0, 0]
start_orientation: [0, 0, 0]
start_velocity: [0.5, 0, 0, 0, 0, 0]
use_quaternion: false
```

Key Parameters

Parameter	Description	Units
`start_location`	Initial [x, y, z] position	meters
`start_orientation`	Initial [roll, pitch, yaw] orientation	radians
`start_velocity`	Initial [u, v, w, p, q, r] velocity	m/s, rad/s
`use_quaternion`	Whether to use quaternions for orientation	boolean

Sensors Configuration (`sensors.yml`)

This file defines the sensors attached to the vessel.

Example from sookshma:

```yaml
# Custom covariance flag is applicable only when using GNC

sensors:
  -
    sensor_type: IMU
    sensor_topic: /sookshma_00/imu/data
    sensor_location: [0.0, 0.0, 0.0]
    sensor_orientation: [0.0, 0.0, 0.0]
    publish_rate: 50
    use_custom_covariance: true
    custom_covariance:
      orientation_covariance: [4.97116638e-07, 1.92100383e-07, -5.37921803e-06, 1.92100383e-0
      linear_acceleration_covariance: [0.01973958, -0.01976063, 0.02346221, -0.01976063, 0.02
      angular_velocity_covariance: [5.28022053e-05, 4.08840955e-05, -1.15368805e-05, 4.088409

  -
    sensor_type: UWB
    sensor_topic: /sookshma_00/uwb
    sensor_location: [0.0, 0.0, 0.0]
    sensor_orientation: [0.0, 0.0, 0.0]
    publish_rate: 1
    use_custom_covariance: true
    custom_covariance:
      position_covariance: [0.04533883, -0.05014115, 0.0, -0.05014115, 0.05869406, 0.0, 0.0,

  -
    sensor_type: encoder
    actuator_type: Rudder
    actuator_id: 1
    sensor_topic: /sookshma_00/Rudder_1/encoder
    publish_rate: 10
```

Key Parameters

Parameter	Description	Units
`sensor_type`	Type of sensor (IMU, GPS, UWB, DVL, encoder, etc.)	string
`sensor_topic`	ROS topic for publishing sensor data	string
`sensor_location`	[x, y, z] position	meters
`sensor_orientation`	[roll, pitch, yaw] orientation	radians
`publish_rate`	Data publication frequency	Hz
`use_custom_covariance`	Whether to use custom noise covariance	boolean
`custom_covariance`	Sensor-specific covariance matrices	various
`actuator_type`	For encoders, the type of actuator	string
`actuator_id`	For encoders, the ID of the actuator	integer

Guidance Configuration (`guidance.yml`)

This file defines waypoints and guidance parameters for autonomous navigation as used in the PID Waypoint Tracking Example.

Example from sookshma:

```yaml
waypoints:
- [0, 0, 0]
- [-4, -10, 0]
- [8, -5, 0]
- [9, 9, 0]
- [0, 0, 0]
waypoints_type: XYZ
```

Key Parameters

Parameter	Description	Units
`waypoints`	List of waypoint coordinates	meters
`waypoints_type`	Coordinate system for waypoints (XYZ, LLA, NED)	string
`lookahead_distance`	Optional: Distance for lookahead-based guidance	meters
`acceptance_radius`	Optional: Radius to consider waypoint reached	meters

Parameter	Description	Units
`guidance_type`	Optional: Type of guidance algorithm	string

NACA Airfoil Data (example: `NACA0015.csv`)

This file contains aerodynamic/hydrodynamic coefficients for the NACA0015 airfoil profile, used for modeling control surfaces like rudders. The file includes:

- `Alpha`: Angle of attack in degrees
- `Cl`: Lift coefficient
- `Cd`: Drag coefficient
- `Cdp`: Pressure drag coefficient
- `Cm`: Pitching moment coefficient
- `Top_Xtr`: Top surface transition location
- `Bot_Xtr`: Bottom surface transition location

Sample excerpt:

```
Alpha,Cl,Cd,Cdp,Cm,Top_Xtr,Bot_Xtr
-19.750,-1.2970,0.09858,0.09490,-0.0142,1.0000,0.0253
-19.500,-1.3072,0.09342,0.08964,-0.0166,1.0000,0.0255
...
0.000,0.0000,0.00632,0.00147,0.0000,0.6107,0.6107
...
19.750,1.2970,0.09858,0.09490,0.0142,0.0253,1.0000
```

This data is used to calculate the forces and moments generated by control surfaces at different angles of attack. This file is obtained from Airfoil Tools website.

Input File Processing

The `read_input.py` module is responsible for loading and processing all input files. The main function is `read_input()`, which:

1. Loads the main simulation configuration file
2. Extracts global simulation parameters
3. Processes each vessel's configuration files
4. Transforms component positions and orientations to be relative to the vessel's coordinate origin (CO)
5. Returns the processed simulation parameters and agent configurations

Key Functions

`read_input(input_file)`

The main entry point that reads the entire configuration:

```
def read_input(input_file: str = None) -> Tuple[Dict, List[Dict]]:
    """Read vessel parameters and configuration from input files.

    Args:
        input_file: Path to the main simulation input file. If None, uses default.

    Returns:
        Tuple containing:
            sim_params (Dict): Global simulation parameters
            agents (List[Dict]): List of agents, each containing vessel config and hydrodynar
    """
```

`load_yaml(file_path)`

Loads a YAML file and returns its contents:

```
def load_yaml(file_path: str) -> Dict:
    """Load a YAML file and return its contents."""
    with open(file_path, 'r') as file:
        return yaml.safe_load(file)
```

`resolve_path(base_path, file_path, name)`

Resolves file paths, replacing `{name}` with the actual vessel name:

```
def resolve_path(base_path: str, file_path: str, name: str) -> str:
    """Resolve a file path, replacing {name} with the actual vessel name."""
    resolved = file_path.replace('{name}', name)
    if not os.path.isabs(resolved):
        resolved = os.path.join(base_path, resolved)
    return resolved
```

`transform_to_co_frame(vessel_config)`

Transforms all component positions and orientations to be relative to the vessel's coordinate origin (CO):

```
def transform_to_co_frame(vessel_config: Dict) -> Dict:
    """Transform all component positions and orientations to be relative to the CO frame."""
    # Implementation details...
```

Cross-Flow Drag Generation

The `read_input` function also initializes a `CrossFlowGenerator` to calculate cross-flow drag coefficients:

```
cross_flow_generator = CrossFlowGenerator(gdf_file, hydra_file, hydrodynamics_file,
                                          initial_conditions_file, agent['type'])
cross_flow_generator.update_yaml_file()
```

This automatically computes and updates the hydrodynamic coefficients for sway-yaw (and heave-pitch for AUVs) using strip theory and Hoerner's cross-flow drag formulation. This happens if the `cross_flow_drag` flag is set to `true` in the `hydrodynamics.yml` file.

Common Customizations

Here are some common customizations you might want to make to the input files:

Changing Vessel Dynamics

To adjust a vessel's dynamic behavior:

1. Modify hydrodynamic coefficients in `hydrodynamics.yml`
2. Adjust mass and inertia properties in `inertia.yml`
3. Change the center of gravity or buoyancy in `geometry.yml`

Adding or Modifying Sensors

To add a new sensor:

1. Add a new sensor configuration to `sensors.yml` with appropriate parameters
2. Set a realistic update rate and noise characteristics
3. Position the sensor at an appropriate location on the vessel

Creating a New Vessel

To create a new vessel type:

1. Create a new folder with the vessel name
2. Copy and modify the configuration files from an existing vessel
3. Adjust all physical parameters to match the new vessel's characteristics
4. Add the new vessel to the `agents` list in `simulation_input.yml`

Example: Complete Vessel Configuration

Here's an example of a complete vessel configuration for a surface vessel:

1. **simulation_input.yml**:

```yaml
sim_time: 100.0
time_step: 0.01
density: 1025.0
gravity: 9.80665
world_size: [1000, 1000, 100]
gps_datum: [13.06, 80.28, 0]
nagents: 1

agents:
  - name: "example_vessel"
    type: "usv"
    active_dof: [1, 1, 0, 0, 0, 1]
    U: 2.0
    maintain_speed: false
    geometry: "inputs/{name}/geometry.yml"
    inertia: "inputs/{name}/inertia.yml"
    hydrodynamics: "inputs/{name}/hydrodynamics.yml"
    control_surfaces: "inputs/{name}/control_surfaces.yml"
```

```
    thrusters: "inputs/{name}/thrusters.yml"
    initial_conditions: "inputs/{name}/initial_conditions.yml"
    sensors: "inputs/{name}/sensors.yml"
    guidance: "inputs/{name}/guidance.yml"
```

2. **geometry.yml**:

```
geometry_file: "inputs/example_vessel/geometry.gdf"
length: 5.0
breadth: 1.5
depth: 1.0

CO:
  position: [0.0, 0.0, 0.0]
  orientation: [0.0, 0.0, 0.0]

CG:
  position: [0.0, 0.0, 0.2]
  orientation: [0.0, 0.0, 0.0]

CB:
  position: [0.0, 0.0, -0.3]
  orientation: [0.0, 0.0, 0.0]

gyration: [1.5, 1.5, 0.8]
```

3. **inertia.yml**:

```
mass: 1000.0
buoyancy_mass: 1000.0
inertia_matrix: "None"
added_mass_matrix: "None"
```

4. **hydrodynamics.yml**:

```
dim_flag: false
hydra_file: "inputs/example_vessel/hydrodynamics.json"
coriolis_flag: true

X_u: -50.0
X_u_au: -100.0
Y_v: -200.0
```

```yaml
Y_v_av: -400.0
Y_r: -50.0
N_v: -50.0
N_r: -300.0
N_r_ar: -400.0
```

5. **control_surfaces.yml**:

```yaml
naca_file: "inputs/naca0015.csv"

control_surfaces:
  - control_surface_id: 1
    control_surface_name: "Rudder"
    control_surface_location: [2.4, 0.0, 0.0]
    control_surface_orientation: [0, 0, 1.57]
    control_surface_area: 0.5
    control_surface_T: 0.5
    control_surface_delta_max: 35.0
    control_surface_deltad_max: 20.0
    control_surface_hydrodynamics: "None"
```

6. **thrusters.yml**:

```yaml
thrusters:
  - thruster_id: 1
    thruster_name: "Main Propeller"
    thruster_location: [-2.4, 0.0, 0.0]
    thruster_orientation: [0.0, 0.0, 0.0]
    D_prop: 0.4
    KT_at_J0: 0.5
    n_max: 1500
    tp: 0.05
```

7. **initial_conditions.yml**:

```yaml
start_location: [0.0, 0.0, 0.0]
start_orientation: [0.0, 0.0, 0.0]
start_velocity: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
use_quaternion: false
```

8. **sensors.yml**:

```yaml
sensors:
  - sensor_type: "IMU"
    sensor_topic: "/vessel/imu"
    sensor_location: [0.0, 0.0, 0.1]
    sensor_orientation: [0.0, 0.0, 0.0]
    publish_rate: 100
    use_custom_covariance: true
    custom_covariance:
      orientation_covariance: [0.01, 0.0, 0.0, 0.0, 0.01, 0.0, 0.0, 0.0, 0.01]
      angular_velocity_covariance: [0.01, 0.0, 0.0, 0.0, 0.01, 0.0, 0.0, 0.0, 0.01]
      linear_acceleration_covariance: [0.05, 0.0, 0.0, 0.0, 0.05, 0.0, 0.0, 0.0, 0.05]
```

9. **guidance.yml**:

```yaml
waypoints:
  - [0.0, 0.0, 0.0]
  - [50.0, 0.0, 0.0]
  - [50.0, 50.0, 0.0]
  - [0.0, 50.0, 0.0]
  - [0.0, 0.0, 0.0]
waypoints_type: XYZ
```